PRICEPILOT — PRODUCTION-GRADE REPAIR, COMPLETION AND QUALITY UPGRADE

Work directly inside the existing repository:

https://github.com/witejackel-eng/pricepilot

Current production deployment:

https://pricepilot-self.vercel.app/

The objective is to upgrade the current PricePilot application from an attractive prototype into a highly reliable, production-quality product-pricing system rated at least **9.5/10** for:

- Financial calculation accuracy
- Excel import reliability
- Pricing recommendation usefulness
- Data integrity
- User experience
- Performance
- Accessibility
- Code quality
- Test coverage
- Deployment safety
- Documentation

Do not rebuild the application from scratch.

Do not replace the current visual identity unnecessarily.

Do not create a new landing page.

Do not spend the first phase polishing charts, animations or decorative styling.

The current interface is already a reasonable foundation. The primary work must be:

1. Repair the financial engine.
2. Repair Excel and CSV import.
3. Eliminate conflicting calculations across screens.
4. Add rigorous automated testing.
5. Improve product-management workflows.
6. Improve Excel export.
7. Improve data storage and performance.
8. Remove scaffold code and incomplete features.
9. Make the application trustworthy enough for real pricing decisions.

The application must remain private and browser-based.

Uploaded business data must not be sent to an external server.

---

1. NON-NEGOTIABLE DEVELOPMENT RULES

1.1 Work inside the existing codebase

Retain the existing:

- Next.js application
- TypeScript
- Tailwind CSS
- shadcn/ui
- Zustand where appropriate
- Recharts
- SheetJS
- Existing PricePilot visual direction
- Existing app navigation where useful

Refactor weak architecture instead of discarding the whole project.

1.2 Do not hide build errors

Remove this configuration immediately:

```
typescript: {
  ignoreBuildErrors: true
}
```

The application must not deploy while TypeScript errors exist.

Add and enforce these scripts:

```
{
  "scripts": {
    "dev": "next dev",
    "typecheck": "tsc --noEmit",
    "lint": "eslint .",
    "test": "vitest run",
    "test:watch": "vitest",
    "test:coverage": "vitest run --coverage",
    "build": "next build",
    "verify": "npm run typecheck && npm run lint && npm run test && npm run
build"
  }
}
```

Use Bun equivalents only if the repository is consistently Bun-based, but the commands must also be documented clearly.

1.3 No fake completion

Do not say a feature is complete unless it:

- Works in the interface
- Uses the real calculation engine
- Has validation
- Has an appropriate test
- Passes TypeScript
- Passes lint
- Passes production build

Do not leave:

- TODO comments
- Placeholder buttons
- Non-functional controls
- Fake progress bars
- Hardcoded chart results
- Silent error handling
- Decorative options that do nothing

1.4 Preserve user data during upgrades

Existing PricePilot browser data may already exist.

Implement:

- Versioned database migrations
- Safe fallback handling
- Backup before destructive migration
- Graceful recovery from malformed legacy data
- Clear notice when data is migrated

Never silently destroy previously stored product data.

---

2. FIRST PRIORITY: REBUILD THE FINANCIAL ENGINE CORRECTLY

The financial calculation engine is the most important part of PricePilot.

Create a single authoritative pricing engine.

No React component may write its own margin, tax, fee or profit formula.

All pages must consume the same engine output.

Create a canonical function such as:

```
calculateOutcomeAtPrice({
  product,
  sellingPrice,
  businessSettings,
  effectiveRule
})
```

It should return a fully typed object such as:

```
interface PriceOutcome {
  enteredSellingPrice: number;
  customerPayableAmount: number;
  netSalesRevenue: number;

  outputTax: number;
  recoverableInputTax: number;
  nonRecoverableInputTax: number;

  purchaseCost: number;
  fixedProductCosts: number;
  expectedReturnCost: number;
  expectedDamageCost: number;
  totalLandedCost: number;

  marketplacePercentageFee: number;
  marketplaceFixedFee: number;
  paymentPercentageFee: number;
  paymentFixedFee: number;
  otherPercentageFees: number;
  otherFixedFees: number;
  totalSellingFees: number;

  totalCostPerSuccessfulSale: number;
  netProfit: number;
  effectiveMarginPercent: number;
  markupPercent: number;

  isProfitable: boolean;
  isBreakEven: boolean;
  satisfiesMinimumMargin: boolean;
  satisfiesTargetMargin: boolean;
  satisfiesMinimumProfit: boolean;
```

```
  warnings: PricingWarning[];
}
```

Every screen must use this function:

- Product table
- Product drawer
- Dashboard
- Price simulator
- Scenario comparison
- Recommendations
- Excel export
- Warning generator
- Pricing explanation

There must never be separate simplified formulas in individual UI files.

---

3. FIX THE MISSING PURCHASE-COST BUG

The existing code can substitute the default shipping cost when purchase cost is missing.

This is unacceptable.

Purchase cost resolution must be:

```
const purchaseCost = product.purchaseCost ?? 0;
```

Do not substitute:

- Shipping cost
- Packaging cost
- A business default
- Existing selling price
- Any unrelated value

If purchase cost is missing or zero:

- Mark the product as `missing-cost`
- Do not produce an approved recommendation
- Display recommendations as unavailable
- Explain why pricing cannot be trusted
- Allow an optional draft estimate only if the user explicitly enables “Estimate with incomplete data”
- Mark any draft estimate with low confidence
- Never export a draft estimate as an approved final price

Add tests proving that missing purchase cost never becomes shipping cost.

---

4. IMPLEMENT A CORRECT GST AND TAX MODEL

PricePilot must support India-focused GST accurately while remaining usable internationally.

4.1 Separate price tax treatment from cost tax treatment

Add clear settings for:

Selling-price tax mode

- Tax inclusive
- Tax exclusive
- Tax exempt

Purchase-cost tax mode

- Cost entered excluding tax
- Cost entered including tax

Input tax credit

- Input tax credit recoverable
- Input tax credit not recoverable
- Partially recoverable

The simple setup can use sensible defaults, while advanced settings expose the full model.

4.2 Tax-inclusive selling price

When a customer-facing selling price includes tax:

```
Net sales revenue = Inclusive price / (1 + tax rate)
Output tax = Inclusive price - Net sales revenue
Customer payable = Inclusive price
```

Example:

```
Selling price including 18% GST: ₹1,180
Net sales revenue: ₹1,000
GST collected: ₹180
Customer payable: ₹1,180
```

The full ₹1,180 must not be treated as business revenue.

4.3 Tax-exclusive selling price

When selling price excludes tax:

```
Net sales revenue = Entered price
Output tax = Entered price × tax rate
Customer payable = Entered price + output tax
```

Example:

```
Tax-exclusive price: ₹1,000
GST: ₹180
Customer payable: ₹1,180
Net sales revenue: ₹1,000
```

GST added on top must not be treated as a percentage selling fee that reduces the entered base price.

4.4 Tax-exempt selling

```
Net sales revenue = Entered price
Output tax = 0
Customer payable = Entered price
```

4.5 Purchase-side tax

When purchase cost includes recoverable GST:

```
Net purchase cost = Gross cost / (1 + purchase tax rate)
Recoverable input tax = Gross cost - Net purchase cost
```

When input tax credit is not recoverable:

```
Full gross purchase cost contributes to landed cost
Recoverable input tax = 0
```

If purchase cost excludes tax:

```
Gross purchase payable = Net cost + input tax
```

Then:

- Recoverable input tax must not increase effective product cost.
- Non-recoverable input tax must increase effective product cost.

4.6 Display tax clearly

Show separately:

- Base selling price
- Customer payable
- Output GST
- Recoverable input GST
- Non-recoverable tax
- Net revenue before income tax

Do not call GST “profit”.

Do not provide tax or accounting advice.

Add a visible note:

“Tax calculations are planning estimates. Confirm final treatment with your accountant or tax adviser.”

4.7 Tax tests

Add automated tests covering:

- 18% GST-inclusive selling price
- 18% GST-exclusive selling price
- Tax-exempt product
- Purchase cost including recoverable GST
- Purchase cost including non-recoverable GST
- Zero tax
- Invalid tax above configured limits
- Margin consistency across inclusive and exclusive modes

---

5. FIX PERCENTAGE-FEE HANDLING

Choose one internal percentage convention and enforce it consistently.

Recommended convention:

```
18% is stored as 18
0.18 is used only as an internal decimal after division by 100
```

Create helpers such as:

```
percentageToDecimal(18) // 0.18
decimalToPercentage(0.18) // 18
```

Never mix the two conventions.

The existing simulator divides percentage fees by 100 twice. Fix this.

For a selling price of ₹1,000 with total percentage fees of 12%:

```
Fee amount must be ₹120
```

Not:

```
₹1.20
```

Add tests for:

- 1%
- 2.5%
- 12%
- 50%
- 99%
- Combined fees
- Fixed plus percentage fees
- Fee totals at or above 100%

If combined fees and requested margin make pricing mathematically impossible:

- Do not return ₹99,999,999 as a normal recommendation.
- Return a structured impossible-result state.
- Display:

“Your selected fees and target margin cannot be achieved because they consume 100% or more of revenue.”

---

6. DEFINE PROFIT, MARGIN AND MARKUP CONSISTENTLY

6.1 Net profit per successful sale

Calculate:

```
Net profit =
Net sales revenue
− total landed cost
− marketplace fees
− payment fees
− other selling fees
```

```
- fixed transaction fees
- non-recoverable tax costs
```

Shipping charged to the customer must be handled explicitly:

- If shipping revenue is included in customer payable, include it in revenue.
- If it is only a reimbursement, show it separately.
- Do not silently ignore customer shipping charges.

6.2 Effective margin

```
Effective margin =
Net profit / net sales revenue × 100
```

Use one documented denominator throughout the application.

When tax is included in the customer-facing price, margin must use net sales revenue excluding tax.

6.3 Markup

```
Markup =
Net profit / total effective product cost × 100
```

Clearly state what PricePilot considers “cost”.

Add tooltips:

“Margin measures profit against net sales revenue.”

“Markup measures profit against the effective cost of selling the product.”

6.4 Never use simplified UI formulas

Remove calculations such as:

```
price - totalLandedCost
```

from UI components when displaying actual profit.

Remove calculations such as:

```
(price - landedCost) / price
```

when fees and tax are omitted.

Use only the canonical engine.

---

7. REBUILD PRICE RECOMMENDATIONS

PricePilot must produce four clearly defined recommendation modes.

7.1 Pure break-even price

The lowest price where:

```
Net profit = 0
```

It must include:

- Landed cost
- Expected returns
- Expected damage
- Fixed fees
- Percentage fees
- Tax treatment
- Non-recoverable tax
- Other selling costs

7.2 Minimum Safe Price

The lowest price satisfying all configured minimums:

- Net profit is positive
- Minimum margin is achieved
- Minimum profit per unit is achieved
- Price is above pure break-even

Add global and rule-level:

- Minimum margin
- Minimum profit amount
- Optional minimum markup

The safe price must satisfy all applicable constraints, not merely one.

7.3 Competitive Price

Use competitor information while never violating the safe price.

Support:

- Match lowest

- Match average
- Match highest
- Price below average by percentage
- Price above average by percentage
- Weighted market price
- Ignore competitor data

If the competitor price is below the safe price:

- Never recommend the competitor price as safe
- Display:

“Market pricing is below your minimum profitable price.”

- Show the gap
- Suggest reducing cost, changing channel or accepting a strategic loss only after explicit user acknowledgement

7.4 Balanced Price

Balanced must consider:

- Target margin
- Target profit
- Market average
- Existing price
- Price-change limits
- Rounding rule
- Maximum acceptable market price

Do not use a fixed 50/50 competitor blend for all businesses.

Add configurable weights:

- Profitability weight
- Competitor weight
- Existing-price stability weight

Default to a clear and documented strategy.

7.5 Premium Price

Consider:

- Premium target margin
- Competitor highest price
- Brand positioning
- Maximum market price
- Maximum permitted price increase

Premium price must not be presented as automatically superior.

7.6 Final selected and approved price

Add separate fields:

```
selectedRecommendationMode
customRecommendedPrice
finalApprovedPrice
priceApprovalStatus
approvedAt
approvedByLabel
```

Do not overwrite `currentSellingPrice` merely because the user clicks a recommendation card.

The proper workflow is:

```
Current price
→ Recommended options
→ Select recommendation
→ Review outcome
→ Approve final price
→ Export approved price
```

Allow an explicit action:

“Apply approved price as current selling price”

This must be separate from selecting a recommendation.

---

8. REVALIDATE PRICES AFTER ROUNDING

Every rounded price must be sent back through `calculateOutcomeAtPrice()`.

Workflow:

```
Calculate raw recommendation
→ Apply requested ending or rounding rule
→ Recalculate full price outcome
→ Verify minimum margin and minimum profit
→ If unsafe, move upward to the next valid rounded price
→ Recalculate again
```

Psychological pricing must never make a safe recommendation unsafe.

Support:

- No rounding
- Nearest whole number
- Nearest 5
- Nearest 10
- End in .99
- End in .95
- End in 9
- End in 49
- End in 99
- Custom step
- Always round upward for safety-constrained prices
- Nearest rounding for non-safety custom exploration

Example:

```
Raw calculated price: ₹1,247
Rule: End in 49
Result: ₹1,249
```

After rounding, show the updated:

- Profit
- Margin
- Markup
- Fees
- Tax
- Customer payable

---

9. ADD PRICING-CONFIDENCE AND DATA-COMPLETENESS LOGIC

Do not pretend every recommendation has equal certainty.

Create a confidence system based on available inputs.

Example:

High confidence

- Purchase cost available
- Selling fees available
- Tax treatment configured
- Shipping and packaging available
- Return assumptions available

- Competitor data recently checked

Medium confidence

- Core costs and fees available
- Some secondary costs missing
- Limited competitor data

Low confidence

- Missing cost components
- Unknown tax treatment
- No fee information
- Old competitor data
- Missing purchase cost

Display:

- Confidence badge
- Missing assumptions
- Last competitor check date
- What would improve confidence

Do not allow low-confidence recommendations to appear as “approved” without confirmation.

---

10. REPAIR THE PRICE SIMULATOR COMPLETELY

The simulator must use the same canonical pricing engine.

Fix:

- Percentage fees being divided by 100 twice
- Total fee amount displaying a decimal percentage as currency
- Margin excluding selling fees
- Profit excluding selling fees
- Target margin input not affecting recommendations
- Tax treatment inconsistencies
- Fixed fees not fully represented
- Recommendations not reflecting simulator assumptions

Simulator inputs must include:

- Purchase cost
- Cost tax mode
- Purchase tax rate
- Input tax credit recoverability
- Shipping cost
- Packaging cost
- Handling cost

- Advertising cost per sale
- Other fixed costs
- Marketplace percentage fee
- Marketplace fixed fee
- Payment percentage fee
- Payment fixed fee
- Other percentage fees
- Other fixed fees
- Return rate
- Return handling cost
- Product recovery rate
- Damage rate
- Selling tax mode
- Selling tax rate
- Minimum margin
- Target margin
- Premium margin
- Minimum profit
- Competitor lowest
- Competitor average
- Competitor highest
- Proposed selling price

Show live outputs:

- Customer payable
- Net sales revenue
- Output tax
- Total landed cost
- Selling fees
- Break-even price
- Minimum safe price
- Competitive price
- Balanced price
- Premium price
- Net profit
- Effective margin
- Markup
- Confidence
- Warnings

Saving a simulator scenario must preserve the simulator assumptions and results.

Do not save an empty product snapshot as a scenario without the simulator data.

Add a dedicated `SimulatorScenario` structure or a scenario type field.

---

11. REPAIR EXCEL AND CSV IMPORT

The import workflow is a core feature and must be dependable.

11.1 Fix the current return-value mismatch

The current importer expects values such as:

```
result.products
result.duplicates
```

while the cleaning function returns different property names.

Create one typed import result interface and use it everywhere:

```
interface CleanImportResult {
  cleanedProducts: ImportedProductDraft[];
  skippedRows: ImportRowIssue[];
  duplicateRows: ImportRowIssue[];
  invalidRows: ImportRowIssue[];
  warnings: ImportRowIssue[];
  statistics: ImportStatistics;
}
```

TypeScript must catch future mismatches.

11.2 Fix stale automatic mapping

Do not call column detection using stale React state.

Use the headers returned directly from the parser:

```
const parsedHeaders = result.sheets[0].headers;
setFileData(...);
setMappings(detectColumnMappings(parsedHeaders));
```

11.3 Validate file before parsing

Support:

- `.xlsx`
- `.xls`
- `.csv`
- `.tsv`

Validate:

- Extension
- MIME type where available
- File size
- Empty file
- Corrupt workbook
- Password-protected workbook
- No usable sheets
- No usable rows

Use a configurable default maximum of 20 MB.

11.4 Allow heading-row selection

The user must be able to choose:

- Row 1 as headings
- Row 2 as headings
- Any row within the first 20 rows

Show a preview before conversion.

11.5 Improve column mapping

Automatically detect variations including:

- Product
- Product Name
- Item
- Item Name
- SKU
- Model
- Product Code
- Cost
- Cost Price
- Purchase Price
- Buying Price
- Unit Cost
- Landed Cost
- Selling Price
- Sale Price
- MRP
- GST
- Tax
- Commission
- Payment Fee
- Shipping
- Freight
- Packaging
- Quantity

- Stock
- Monthly Sales
- Competitor Price

Allow:

- Manual mapping
- Skipping columns
- Saving mapping templates
- Reusing templates for later imports
- Mapping one source field only once unless explicitly allowed
- Validation that required columns exist

11.6 Required fields

Use a consistent policy:

Required:

- Product name or SKU
- Purchase cost for trustworthy recommendations

SKU must be recommended but not universally mandatory.

Rows with product identity but missing cost may be imported as `missing-cost`, but must not receive approved recommendations.

11.7 Percentage interpretation

Support:

- `18`
- `18%`
- `0.18`

Add an import-level setting:

```
Percentage format:
- Detect automatically
- Whole percentages: 18 means 18%
- Decimal fractions: 0.18 means 18%
```

Auto-detection must use the entire column distribution rather than guessing from a single cell.

11.8 Make cleaning options functional

The existing checkboxes must control real behaviour.

Support:

- Strip currency symbols
- Remove grouping commas
- Parse parentheses as negative
- Parse percentage symbols
- Replace blank values with defaults
- Import and flag incomplete rows
- Skip incomplete rows
- Keep duplicate rows
- Skip duplicate rows
- Merge duplicates
- Update existing product by SKU
- Append as a new product
- Stop import on critical errors

The selected options must be passed to the cleaning engine.

11.9 Existing-product reconciliation

On repeated import, ask:

```
How should matching SKUs be handled?
- Update existing products
- Keep existing values
- Create duplicates
- Review each conflict
```

Show before/after differences.

11.10 Preserve original spreadsheet data

Store:

- Original row number
- Original sheet name
- Original unmapped columns
- Source file name
- Import date
- Import batch ID

This makes it possible to export an updated version without losing user columns.

11.11 Import report

After processing, show:

- Valid products
- Incomplete products

- Invalid products
- Duplicate SKUs
- Updated products
- New products
- Skipped rows
- Missing costs
- Missing prices
- Invalid percentages
- Estimated inventory cost where quantity exists

Allow download of:

- Import error report
- Skipped rows file
- Duplicate review file

---

12. REBUILD PRODUCT MANAGEMENT

12.1 Implement Add Product

The current Add Product button must be fully functional.

Create an Add Product form with:

- Basic mode
- Advanced mode
- Validation
- Live pricing preview
- Save and add another
- Cancel
- Duplicate existing product

12.2 Product editing

Allow editing:

- Product name
- SKU
- Category
- Brand
- Supplier
- Purchase cost
- Cost tax treatment
- Shipping
- Packaging
- Handling
- Advertising
- Returns

- Damage
- Tax
- Marketplace fees
- Payment fees
- Competitor prices
- Sales channel
- Existing price
- Quantity
- Expected monthly sales
- Notes

12.3 Inline editing

Allow safe inline editing for:

- Purchase cost
- Existing selling price
- Shipping
- Packaging
- Marketplace fee
- Payment fee
- Target margin
- Monthly units
- Final approved price

Recalculate immediately after an edit.

Provide undo for recent edits.

12.4 Bulk actions

Implement:

- Apply category
- Apply brand
- Apply channel
- Apply tax treatment
- Apply pricing rule
- Set target margin
- Set minimum profit
- Set marketplace fee
- Set payment fee
- Set recommendation mode
- Approve selected prices
- Mark for review
- Archive
- Delete
- Export selected

12.5 Product lifecycle

Support statuses:

- Active
- Draft
- Missing data
- Needs review
- Approved
- Archived

Do not permanently delete products without confirmation.

---

13. FIX THE PRODUCT DETAIL DRAWER

Every recommendation card must use the canonical pricing engine.

For each recommendation display:

- Customer payable
- Net sales revenue
- Tax
- Total costs
- Total selling fees
- Net profit
- Effective margin
- Markup
- Difference from current price
- Difference from competitor average
- Confidence level
- Warnings

Add:

- Current price outcome
- Pure break-even outcome
- Minimum safe outcome
- Competitive outcome
- Balanced outcome
- Premium outcome
- Custom price outcome

Custom price must update all values instantly.

Add a clear approval workflow:

```
Select recommendation
→ Review full outcome
```

```
→ Approve price
→ Optionally apply it as current price
```

Generate a plain-language explanation from structured rules.

Example:

“Your current GST-inclusive price of ₹1,180 produces net sales revenue of ₹1,000. After landed cost and selling fees, estimated profit is ₹120 per unit, giving an effective margin of 12%. Your minimum target is 20%, so the recommended safe price is ₹1,349.”

Do not use external AI for this explanation.

---

14. FIX RULE PRIORITY AND RULE MERGING

Use one documented priority order everywhere:

```
Product
> Brand
> Category
> Channel
> Global
```

Or choose another order, but the engine, interface, README and help text must all match.

Prefer the order above unless there is a strong technical reason otherwise.

14.1 Support inherited rule merging

Do not simply select one entire rule and ignore all lower-level defaults.

Merge rules field by field.

Example:

- Global rule defines tax and rounding
- Category rule defines margin
- Brand rule defines premium margin
- Product rule defines minimum profit

The effective rule should inherit all applicable fields.

Create:

```
resolveEffectivePricingPolicy(product, rules, businessSettings)
```

Return:

- Effective value for every setting
- Source rule for every value
- Rule-resolution trace

Display:

“Target margin: 25% from CCTV Category Rule”

“Rounding: End in 99 from Global Rule”

14.2 Conflict detection

Detect:

- Same-level rules targeting the same entity
- Equal-priority conflicts
- Impossible margin and fee combinations
- Product rule referencing a missing product
- Category rule referencing no products
- Disabled parent rules
- Contradictory price caps

Allow the user to resolve conflicts.

---

15. ADD REAL SALES-VOLUME SUPPORT

PricePilot currently adds per-unit profit across products and labels it estimated profit.

That is misleading.

Add:

- Current stock quantity
- Monthly units sold
- Expected monthly units
- Optional annual units
- Sales period
- Inventory age

Calculate:

```
Projected monthly revenue
Projected monthly profit
```

```
Projected annual profit
Inventory investment
Potential profit improvement
Stock-at-risk value
```

When quantity or sales volume is unavailable:

- Label the metric “Profit per one unit of each selected product”
- Do not call it monthly or total estimated profit

Dashboard metrics must clearly identify whether they are:

- Per unit
- Monthly
- Annual
- Inventory-based

---

16. REBUILD THE DASHBOARD USING CORRECT OUTCOMES

All dashboard values must use canonical engine outcomes.

Include:

- Total products
- Products with complete data
- Products missing cost
- Products below break-even
- Products below minimum margin
- Average current effective margin
- Average approved-price margin
- Current projected monthly profit
- Approved-price projected monthly profit
- Potential monthly profit improvement
- Inventory value
- High-risk stock value
- Products where competitor price is below safe price
- Products with low-confidence recommendations

Filters:

- Category
- Brand
- Channel
- Status
- Import batch
- Confidence
- Tax treatment

- Date range

Charts:

- Profitability distribution
- Current versus approved margin
- Projected profit improvement by category
- Margin versus sales-volume scatter plot
- Products requiring a price increase
- Products where price increase may exceed the market
- Inventory value by profitability status

Charts must not recalculate financial formulas independently.

---

17. IMPROVE SAVED SCENARIOS

Scenario types:

- Full catalogue snapshot
- Pricing-rule scenario
- Supplier-cost scenario
- Marketplace-fee scenario
- Tax scenario
- Simulator scenario
- Discount scenario

Store:

- Scenario type
- Products
- Rules
- Settings
- Simulator assumptions where applicable
- Created date
- Notes
- Comparison baseline
- Schema version

Comparison must show:

- Revenue difference
- Profit difference
- Margin difference
- Customer-price difference
- Tax difference
- Number of affected products
- Products becoming unprofitable
- Products becoming safe
- Monthly profit effect where volume exists
- Inventory risk effect

Use deep copies or immutable serialisation.

Do not store mutable references.

---

18. REBUILD EXCEL EXPORT

The export must be suitable for real business use.

18.1 Preserve the original data

When products came from a spreadsheet:

- Preserve original columns
- Preserve original ordering
- Preserve row reference
- Append PricePilot columns
- Do not discard unmapped business columns

18.2 Export the approved price

Do not always export Balanced Price.

Export:

- Current selling price
- Minimum safe price
- Competitive price
- Balanced price
- Premium price
- Selected recommendation
- Final approved price
- Approval status

18.3 Export presets

Implement:

Simple Price List

- Product
- SKU
- Existing price
- Approved price
- Price change

Full Pricing Analysis

- All input costs

- All fee assumptions
- Tax treatment
- All recommendations
- Profit
- Margin
- Markup
- Confidence
- Warnings
- Applied rules

Ecommerce Update

- SKU
- Product
- MRP
- Selling price
- Discount
- Stock
- Tax rate

Accountant Report

- Gross sales
- Net sales revenue
- Output tax
- Recoverable input tax
- Non-recoverable input tax
- Landed cost
- Selling fees
- Profit
- Margin

Marketplace Upload

Allow a configurable mapping preset for marketplace templates without hardcoding a single platform’s format.

18.4 Workbook sheets

Create:

1. Products
2. Pricing Summary
3. Products Requiring Review
4. Applied Pricing Rules
5. Import Issues
6. Export Information

18.5 Excel formatting

Add:

- Bold header rows
- Freeze panes
- Autofilters
- Correct currency number formats
- Correct percentage formats
- Sensible column widths
- Wrapped warnings
- Conditional formatting where supported
- Data validation where useful
- Export timestamp
- Currency and tax assumptions
- PricePilot schema version

18.6 Export scope

Implement fully:

- All products
- Current filtered products
- Selected products
- Approved products
- Products requiring review
- Products imported in a selected batch

The “filtered products” option must use the actual current filters.

Show a success toast and a useful error dialog.

---

19. IMPROVE DATA STORAGE

Do not store large product catalogues and full scenarios only in localStorage.

Use:

- IndexedDB for products
- IndexedDB for scenarios
- IndexedDB for import batches
- LocalStorage for small user preferences
- Versioned migrations

Dexie may be used as an IndexedDB wrapper.

Create repositories such as:

```
src/db/
  database.ts
  migrations.ts
  products-repository.ts
  scenarios-repository.ts
  imports-repository.ts
  settings-repository.ts
```

Include:

- Transactional writes
- Batch import
- Safe rollback
- Storage quota handling
- Backup export
- Backup import
- Data integrity validation

Display a visible error if the browser cannot save data.

Do not silently fail after a storage-quota error.

---

20. PERFORMANCE FOR LARGE PRODUCT LISTS

The app should handle at least 5,000 products smoothly.

Use:

- IndexedDB
- Virtualised product table
- Memoised selectors
- Batch recalculation
- Debounced search
- Web Worker for heavy Excel parsing
- Web Worker for bulk pricing calculations where useful
- Progress based on actual completed rows
- Cancellation for large imports
- Efficient scenario serialisation

Test with generated datasets of:

- 100 products
- 1,000 products
- 5,000 products

Measure:

- Import duration

- Table responsiveness
- Filter responsiveness
- Recalculation duration
- Export duration

Document practical browser limits honestly.

---

21. VALIDATION AND ERROR STATES

Use Zod or equivalent typed validation.

Validate:

- Purchase cost
- Selling price
- Fees
- Margin targets
- Minimum profit
- Tax rates
- Quantities
- Sales volume
- Competitor prices
- Rounding settings
- Import mappings
- Scenario data
- Backup data

Examples:

- “Purchase cost cannot be negative.”
- “Target margin must be below 100%.”
- “Total percentage fees and margin make this price mathematically impossible.”
- “Purchase cost is required before a price can be approved.”
- “The selected tax treatment requires a tax rate.”
- “This SKU already exists.”
- “No usable rows were found in this sheet.”
- “The approved price no longer satisfies the minimum margin after costs changed.”

Do not expose raw stack traces to users.

Provide:

- Retry
- Return
- Download error report
- Restore backup
- Reset only the affected operation

---

22. AUTOMATED TESTING

Install:

- Vitest
- React Testing Library
- user-event
- jsdom
- Coverage tooling
- Playwright for end-to-end tests

22.1 Financial engine tests

Test:

- Base cost
- Landed cost
- Expected return cost
- Damage cost
- Fixed fees
- Percentage fees
- Break-even
- Minimum safe price
- Competitive price
- Balanced price
- Premium price
- Minimum profit
- Margin
- Markup
- GST inclusive
- GST exclusive
- Input tax credit
- Non-recoverable tax
- Customer shipping revenue
- Rounding
- Impossible combinations
- Missing cost
- Zero price
- Negative input
- Competitor below safe price
- Price cap conflict

22.2 Required calculation examples

Example A

```
Purchase cost: ₹1,000
Shipping: ₹100
Packaging: ₹50
```

```
Percentage selling fees: 12%
Target effective margin: 20%
Tax: exempt
```

Before rounding:

```
Required target price = ₹1,150 / (1 − 0.12 − 0.20)
Required target price ≈ ₹1,691.18
```

The test must accept only a result within a defined rounding tolerance.

Example B

```
Purchase cost: ₹500
Other fixed cost: ₹50
Current selling price: ₹600
Percentage selling fees: 15%
Tax: exempt
```

Expected:

```
Percentage fees: ₹90
Total cost and fees: ₹640
Net profit: -₹40
Effective margin: approximately -6.67%
Status: loss-making
```

Example C

```
GST-inclusive price: ₹1,180
GST rate: 18%
```

Expected:

```
Net sales revenue: ₹1,000
Output GST: ₹180
Customer payable: ₹1,180
```

Example D

```
Tax-exclusive selling price: ₹1,000
GST rate: 18%
```

Expected:

```
Net sales revenue: ₹1,000
Output GST: ₹180
Customer payable: ₹1,180
```

Example E

```
Minimum safe price: ₹1,499
Competitor average: ₹1,399
```

Expected:

- Competitive safe recommendation must not be below ₹1,499.
- A market-conflict warning must appear.

Example F

```
Raw recommended price: ₹1,247
Rounding rule: End in 49
```

Expected:

```
Rounded price: ₹1,249
```

Then recalculate the full outcome at ₹1,249.

22.3 Import tests

Test:

- XLSX
- XLS
- CSV
- TSV
- Multiple sheets
- Heading row 1
- Heading row 3
- Currency symbols
- Indian comma grouping
- Western comma grouping
- Percentages as 18
- Percentages as 18%
- Percentages as 0.18
- Blank cells
- Duplicate SKUs

- Invalid numeric values
- Corrupt workbook
- Empty workbook
- Existing-product merge
- Original-column preservation

22.4 Export tests

Test:

- Valid XLSX output
- Valid CSV output
- Original columns retained
- Calculated columns appended
- Approved price exported
- Correct sheets created
- Correct number formats
- Filtered export
- Selected export
- Review export

22.5 UI integration tests

Test:

- Onboarding
- Add product
- Edit product
- Import flow
- Mapping flow
- Cleaning options
- Recommendation selection
- Price approval
- Rule creation
- Rule conflict
- Simulator
- Scenario save
- Scenario comparison
- Export
- Backup and restore

22.6 End-to-end tests

Use Playwright for:

1. Complete onboarding.
2. Upload a test workbook.
3. Map columns.
4. Import products.
5. Review a loss-making product.

6. Apply a safe recommendation.
7. Approve price.
8. Export the workbook.
9. Refresh the browser.
10. Confirm data persists.

---

23. CI AND DEPLOYMENT SAFETY

Add GitHub Actions.

Create:

```
.github/workflows/verify.yml
```

Run on:

- Pull requests
- Push to main

Required jobs:

```
npm ci
npm run typecheck
npm run lint
npm run test
npm run build
```

The workflow must fail on:

- Type errors
- Lint errors
- Test failures
- Build failures

Do not allow production deployment to rely on skipped type validation.

Add optional Playwright smoke testing against preview deployments where practical.

---

24. REMOVE UNUSED AND SCAFFOLD CODE

Audit dependencies and remove anything not used.

Review candidates including:

- Prisma
- NextAuth
- Z.ai SDK
- MDX Editor
- Unused React Query setup
- Unused drag-and-drop packages
- Unused API routes
- Unused database commands
- Unused components
- Dead utilities

Remove the placeholder API response:

```
Hello, world!
```

Remove unnecessary serverless functions if the app is fully client-side.

Update `package.json` to accurately represent the application.

Run a dependency audit.

Do not add a backend merely because unused backend packages already exist.

---

25. REPLACE ALL Z.AI SCAFFOLD BRANDING

Replace:

- Page title
- Description
- Author
- Keywords
- Open Graph metadata
- Twitter metadata
- Favicon
- External Z.ai logo
- Placeholder URLs
- Scaffold comments

Use PricePilot branding.

Suggested metadata:

```
Title: PricePilot — Product Pricing and Profit Optimiser

Description: Import product spreadsheets, calculate landed costs, compare
```

```
pricing strategies and export profitable selling prices privately in your
browser.
```

Create a simple local PricePilot favicon.

Do not claim “AI-powered” unless an actual AI feature exists.

---

26. USER EXPERIENCE POLISH

After calculations and workflows are correct, improve visual usability.

26.1 Onboarding

Provide:

- Quick setup
- Advanced setup
- Skip with safe defaults
- Clear explanation of tax mode
- Marketplace fee presets that users can edit
- Data privacy notice

Do not require users to understand accounting formulas.

26.2 Basic and advanced modes

Basic:

- Purchase cost
- Existing price
- Shipping
- Fees
- Tax mode
- Target margin

Advanced:

- Input tax credit
- Returns
- Damage
- Advertising
- Multiple competitor prices
- Fixed fees
- Minimum profit
- Volume projections
- Rule tracing

26.3 Accessibility

Implement:

- Keyboard navigation
- Visible focus states
- Proper labels
- Accessible dialogs
- Screen-reader announcements
- Non-colour status indicators
- Sufficient contrast
- Mobile-friendly tables or cards
- Reduced-motion support

26.4 Mobile experience

Ensure:

- Sidebar works
- Import mapping is usable
- Product cards replace very wide tables where necessary
- Recommendation cards remain readable
- Dialogs do not overflow
- Buttons have appropriate touch targets

---

27. HELP AND TRANSPARENCY

Add a Help and Methodology section.

Explain:

- Landed cost
- Break-even
- Margin
- Markup
- Net sales revenue
- Tax-inclusive versus tax-exclusive prices
- Input tax credit
- Marketplace fees
- Minimum safe price
- Competitive price
- Balanced price
- Premium price
- Confidence level
- Projected versus actual profit

Add a “How this number was calculated” option next to important metrics.

Show calculation components without forcing users to read formulas.

Provide an audit trail:

```
Purchase cost: ₹X
Shipping: ₹Y
Selling fees: ₹Z
Tax: ₹A
Net revenue: ₹B
Profit: ₹C
Margin: D%
```

---

28. SECURITY AND PRIVACY

Maintain client-side privacy.

Confirm:

- No product data is sent to external servers
- No analytics includes product names, costs or prices
- No external AI API processes spreadsheets
- No hidden upload endpoint exists
- Spreadsheet parsing happens locally
- Export happens locally

Add a privacy indicator:

“Processed locally in your browser.”

Use a strict Content Security Policy where compatible.

Sanitise:

- Spreadsheet cell strings
- Imported notes
- Product descriptions
- Export filenames
- Scenario names

Protect against spreadsheet formula injection in CSV and XLSX exports.

Cells beginning with:

- `=`
- `+`
- `-`
- `@`

must be escaped when they originate from untrusted imported text.

---

29. README AND DOCUMENTATION

Rewrite the README so it describes what is actually implemented.

Include:

- Product overview
- Screenshots
- Technology stack
- Local setup
- Development commands
- Testing commands
- Architecture
- Pricing-engine methodology
- Tax assumptions
- Import behaviour
- Export behaviour
- Local privacy model
- IndexedDB schema
- Backup and restore
- Deployment
- Known limitations
- Financial disclaimer
- Contribution guidance

Do not state that tests exist unless they are included and passing.

Do not claim true optimal-market pricing without sales-response data.

Use correct language:

“Rule-based, market-aware pricing recommendations”

Not:

“Guaranteed optimal pricing”

---

30. DEFINITION OF A 9.5/10 RESULT

The upgraded application will qualify only when all of the following are true.

Financial reliability

- All screens use one engine.

- GST is handled correctly.
- Fees are consistent.
- Missing cost cannot produce a trusted recommendation.
- Rounded prices are revalidated.
- Competitor prices cannot override profitability constraints.
- Minimum profit is supported.
- Calculation tests pass.

Import reliability

- Excel and CSV imports work.
- Auto-mapping works on first upload.
- Cleaning options affect behaviour.
- Duplicate handling works.
- Existing SKU updates work.
- Original columns are preserved.
- Error reports can be downloaded.

Product workflow

- Add Product works.
- Product editing works.
- Bulk actions work.
- Recommendations can be selected.
- Prices can be approved.
- Approved price is stored separately.
- Final approved prices export correctly.

Dashboard reliability

- No dashboard figure ignores fees or tax.
- Projected profit uses units.
- Per-unit figures are labelled honestly.
- Filters work.

Engineering

- No ignored TypeScript errors.
- No broken imports.
- No TODO buttons.
- Automated tests exist.
- CI exists.
- Production build passes.
- Large datasets use IndexedDB.
- 5,000-product testing is documented.

Product polish

- Z.ai branding is removed.
- Metadata is correct.

- Mobile layouts work.
- Accessibility is implemented.
- Error messages are useful.
- Privacy is clear.
- README is accurate.

---

31. REQUIRED IMPLEMENTATION ORDER

Follow this exact order.

Phase 1 — Safety baseline

1. Create a working branch.
2. Add typecheck and tests.
3. Remove ignored TypeScript errors.
4. Fix all existing compile-time errors.
5. Add CI.
6. Remove misleading build configuration.

Phase 2 — Financial engine

1. Create canonical outcome engine.
2. Fix purchase-cost fallback.
3. Implement percentage convention.
4. Implement GST and tax model.
5. Implement true profit, margin and markup.
6. Implement impossible-state handling.
7. Add comprehensive engine tests.

Do not proceed until tests pass.

Phase 3 — Recommendations

1. Rebuild break-even.
2. Rebuild minimum safe.
3. Rebuild competitive.
4. Rebuild balanced.
5. Rebuild premium.
6. Add minimum profit.
7. Add revalidation after rounding.
8. Add confidence.
9. Add recommendation tests.

Phase 4 — Replace UI calculations

1. Product drawer
2. Product table
3. Simulator

4. Dashboard
5. Scenarios
6. Warnings
7. Exports

Delete all duplicated financial formulas from components.

Phase 5 — Import repair

1. Fix result-type mismatches.
2. Fix stale mapping.
3. Add file validation.
4. Add heading selection.
5. Connect cleaning options.
6. Add percentage-mode detection.
7. Add duplicate and merge handling.
8. Preserve original data.
9. Add import tests.

Phase 6 — Product workflow

1. Add Product
2. Full edit
3. Inline edit
4. Bulk actions
5. Approval workflow
6. Sales volume
7. Product lifecycle states

Phase 7 — Export

1. Preserve original columns.
2. Export approved prices.
3. Add presets.
4. Add workbook formatting.
5. Add all requested sheets.
6. Add export tests.

Phase 8 — Storage and performance

1. IndexedDB
2. Migrations
3. Large-import worker
4. Virtualised table
5. Scenario optimisation
6. 5,000-product test

Phase 9 — Product polish

1. Metadata

2. Branding
3. Accessibility
4. Mobile
5. Help
6. Privacy
7. README
8. Dependency cleanup

Phase 10 — Verification

Run:

```
npm run typecheck
npm run lint
npm run test
npm run test:coverage
npm run build
npm run verify
```

Run Playwright end-to-end tests.

Deploy a preview.

Test the production-like flow manually.

Only then update production.

---

32. FINAL MANUAL ACCEPTANCE TEST

Perform this complete workflow before declaring completion:

1. Open PricePilot in a clean browser.
2. Complete onboarding for an Indian business.
3. Select INR and 18% GST.
4. Configure tax-inclusive selling prices.
5. Configure recoverable input tax.
6. Upload a multi-sheet Excel workbook.
7. Select the correct sheet.
8. Select a non-first heading row.
9. Map product, SKU, cost, price, tax and fee fields.
10. Import percentages containing 18, 18% and 0.18.
11. Handle duplicate SKUs.
12. Update an existing SKU.
13. Import a product with missing cost.
14. Confirm it receives no trusted price.
15. Open a loss-making product.
16. Confirm current profit includes tax and fees.

17. Compare all four recommendations.
18. Enter a custom price.
19. Approve a safe price.
20. Confirm approval does not silently overwrite the current price.
21. Apply the approved price explicitly.
22. Add monthly units.
23. Confirm dashboard monthly profit updates.
24. Create a pricing rule.
25. Confirm the effective-rule trace.
26. Create two scenarios.
27. Compare the scenarios.
28. Export the full workbook.
29. Confirm original columns remain.
30. Confirm approved price exports.
31. Confirm all expected sheets exist.
32. Refresh the browser.
33. Confirm all data persists.
34. Export a backup.
35. Clear the application.
36. Restore the backup.
37. Confirm restored data is correct.
38. Test the application on mobile width.
39. Test keyboard navigation.
40. Confirm no console errors.

---

33. FINAL RESPONSE REQUIRED FROM Z.AI

After completing the work, provide a truthful implementation report containing:

1. Summary of completed changes
2. Critical bugs fixed
3. Financial formulas implemented
4. Tax assumptions implemented
5. Import improvements
6. Export improvements
7. Storage changes
8. Performance results
9. Test count
10. Test coverage
11. Typecheck result
12. Lint result
13. Production build result
14. End-to-end test result
15. Remaining limitations
16. Files added
17. Files materially changed
18. Dependencies removed
19. Dependencies added
20. Deployment URL

21. Exact commit SHA

Include command output summaries.

Do not say “all tests pass” unless the test command was actually executed successfully.

Do not say the app supports 5,000 products unless that dataset was actually tested.

Do not claim the application finds the mathematically optimal market price.

Describe it accurately as:

“A private, rule-based and market-aware pricing and profit-analysis application.”